

Završni projekt iz informatike

Kartaška igra inspirirana igrom *Slay the Spire*

Izrađeno u: **Godot Engine 4**

Baza podataka: **SQLite**

15. svibnja 2026.

Sadržaj

1	Opis projekta	2
1.1	Tehnologije	2
2	Struktura projekta	2
3	Baza podataka	2
3.1	Shema baze podataka	2
3.2	Opis tablica	3
3.3	Unos testnih podataka	3
4	GDScript skripte	4
4.1	Card.gd	4
4.2	Player.gd	4
4.3	Protivnik.gd	5
4.4	Main.gd	5
4.4.1	Varijable i inicijalizacija	5
4.4.2	Obrada korisničkog unosa	6
4.4.3	Game loop — faza igrača	7
4.4.4	Game loop — faza protivnika	8
4.4.5	Napad i kraj igre	9
4.4.6	Učitavanje podataka iz baze	9
5	Tijek igre	11
6	Zaključak	12

Opis projekta

Ovaj projekt je kartaška igra za jednog igrača izrađena u **Godot Engine 4** programskom okruženju. Igra je inspirirana popularnom roguelike deck-building igrom *Slay the Spire*. Cilj igre je poraziti protivnika koristeći karte iz svog špila, pri čemu svaka karta troši određenu količinu energije. Igra je povezana s **SQLite** bazom podataka iz koje se dinamički učitavaju svi podaci — korisnici, karte, špilovi i protivnici.

Tehnologije

- **Godot Engine 4** — game engine i skriptiranje (GDScript)
- **SQLite** — relacijska baza podataka za pohranu podataka igre
- **GDScript** — programski jezik specifičan za Godot

Struktura projekta

Projekt se sastoji od sljedećih datoteka:

Datoteka	Opis
Card.gd	Resource klasa koja predstavlja jednu kartu
Player.gd	Klasa koja predstavlja igrača
Protivnik.gd	Klasa koja predstavlja protivnika
Main.gd	Glavna skripta — game loop i logika igre
game.db	SQLite baza podataka

Baza podataka

Shema baze podataka

Baza podataka sastoji se od pet tablica koje zajedno opisuju sve entitete igre.

```
1 CREATE TABLE korisnik (  
2     id          INTEGER PRIMARY KEY AUTOINCREMENT,  
3     username    TEXT,  
4     email       TEXT  
5 );  
6  
7 CREATE TABLE protivnik (  
8     id          INTEGER PRIMARY KEY AUTOINCREMENT,  
9     naziv       TEXT,  
10    hp          INTEGER,  
11    difficulty   TEXT  
12 );  
13  
14 CREATE TABLE karta (  
15     id          INTEGER PRIMARY KEY AUTOINCREMENT,  
16     naziv       TEXT,  
17     attack      INTEGER,  
18     cost        INTEGER
```

```

19 );
20
21 CREATE TABLE deck (
22     id            INTEGER PRIMARY KEY AUTOINCREMENT,
23     naziv         TEXT,
24     vlasnik_tip   TEXT,
25     korisnik_id   INTEGER,
26     protivnik_id  INTEGER
27 );
28
29 CREATE TABLE deck_karta (
30     id            INTEGER PRIMARY KEY AUTOINCREMENT,
31     deck_id       INTEGER,
32     karta_id      INTEGER,
33     kolicina      INTEGER,
34
35     FOREIGN KEY(deck_id) REFERENCES deck(id),
36     FOREIGN KEY(karta_id) REFERENCES karta(id)
37 );

```

Listing 1: Kreiranje tablica baze podataka

Opis tablica

- **korisnik** — pohranjuje podatke o igraču (korisničko ime i email)
- **protivnik** — pohranjuje protivnike s njihovim HP-om i težinom
- **karta** — sve dostupne karte u igri s napadom i cijenom energije
- **deck** — špil koji pripada ili korisniku ili protivniku
- **deck_karta** — vezna tablica koja definira koje karte i u kojoj količini se nalaze u špilu

Unos testnih podataka

```

1  -- Korisnik
2  INSERT INTO korisnik (username, email) VALUES
3      ('Ivan', 'ivan@gmail.com');
4
5  -- Protivnici
6  INSERT INTO protivnik (naziv, hp, difficulty) VALUES
7      ('Goblin', 20, 'easy'),
8      ('Vitez', 35, 'medium'),
9      ('Zmaj', 50, 'hard');
10
11 -- Karte
12 INSERT INTO karta (naziv, attack, cost) VALUES
13     ('Mac', 5, 1),
14     ('Vatrogasna', 8, 2),
15     ('Munja', 12, 3),
16     ('Udarac', 3, 1),

```

```

17      ('Eksplozija', 15, 3),
18      ('Strijela', 6, 2);
19
20  -- Deck za playera i protivnika
21  INSERT INTO deck (naziv, vlasnik_tip, korisnik_id, protivnik_id)
22  VALUES
23      ('Player Deck', 'korisnik', 1, NULL),
24      ('Goblin Deck', 'protivnik', NULL, 1);
25
26  -- Karte u player decku (deck_id = 1)
27  INSERT INTO deck_karta (deck_id, karta_id, kolicina) VALUES
28      (1, 1, 2),
29      (1, 2, 2),
30      (1, 3, 2),
31      (1, 4, 2),
32      (1, 5, 1),
33      (1, 6, 2);
34
35  -- Karte u goblin decku (deck_id = 2)
36  INSERT INTO deck_karta (deck_id, karta_id, kolicina) VALUES
37      (2, 1, 3),
38      (2, 4, 3),
39      (2, 6, 2);

```

Listing 2: INSERT naredbe za testne podatke

GDScript skripte

Card.gd

Klasa `Card` nasljeđuje `Resource` što znači da se instance mogu pohraniti i učitavati kao resursi u Godotu. Svaka karta ima jedinstveni identifikator, naziv, vrijednost napada i cijenu energije.

```

1  extends Resource
2  class_name Card
3
4  var id      : int
5  var naziv   : String
6  var attack  : int
7  var cost    : int

```

Listing 3: Card.gd — klasa karte

Player.gd

Klasa `Player` nasljeđuje `Node`. Igrač ima životne bodove, energiju te dva polja za karte — `deck` (špil iz kojeg vuče) i `hand` (karte u ruci). Metoda `draw_card()` uzima prvu kartu iz špila i stavlja je u ruku.

```

1  extends Node
2  class_name Player

```

```

3
4 var hp      : int    = 20
5 var energy  : int    = 3
6 var deck    : Array  = []
7 var hand    : Array  = []
8
9 func draw_card():
10     if deck.size() > 0:
11         var card = deck.pop_front()
12         hand.append(card)

```

Listing 4: Player.gd — klasa igrača

Protivnik.gd

Klasa `Enemy` nasljeđuje `Node`. Protivnik ima iste attribute kao igrač uz dodatak polja `difficulty` koje označava težinu. Metoda `play_card()` automatski izvlači i vraća prvu kartu iz ruke.

```

1 extends Node
2 class_name Enemy
3
4 var hp      : int    = 20
5 var difficulty : String = "easy"
6 var deck    : Array  = []
7 var hand    : Array  = []
8
9 func draw_card():
10     if deck.size() > 0:
11         var card = deck.pop_front()
12         hand.append(card)
13
14 func play_card():
15     if hand.size() > 0:
16         return hand.pop_front()
17     return null

```

Listing 5: Protivnik.gd — klasa protivnika

Main.gd

Glavna skripta upravlja cjelokupnom logikom igre. Učitava podatke iz baze, pokreće game loop i obrađuje korisnički unos putem tipkovnice.

4.4.1 Varijable i inicijalizacija

```

1 extends Node
2
3 var player      : Player
4 var enemy       : Enemy
5 var game_over   : bool   = false
6 var player_turn : bool   = true

```

```

7 var waiting_for_input      : bool      = false
8 var player_input_received: int        = -1
9 var player_username       : String    = "Igrac"
10 var player_email         : String    = ""
11
12 func _ready():
13     player = Player.new()
14     enemy  = Enemy.new()
15
16     load_player_from_database()
17     load_cards_from_database()
18     load_enemy_from_database()
19
20     for i in range(3):
21         player.draw_card()
22         enemy.draw_card()
23
24     print("=== IGRA POCINJE ===")
25     print("Dobrodošao, ", player_username, "!")
26     print("Player HP: ", player.hp, " | Enemy HP: ", enemy.hp)
27
28     player_phase()

```

Listing 6: Main.gd — varijable i _ready()

4.4.2 Obrada korisničkog unosa

Umjesto čitanja iz konzole, igra koristi Godotov `_input()` callback koji se poziva svaki put kada igrač pritisne tipku. Zastavica `waiting_for_input` osigurava da se tipke ignoriraju kada nije na redu odabir karte.

```

1 func _input(event):
2     if not waiting_for_input:
3         return
4
5     if event is InputEventKey and event.pressed:
6         var key = event.keycode
7
8         if key == KEY_0: player_input_received = 0
9         elif key == KEY_1: player_input_received = 1
10        elif key == KEY_2: player_input_received = 2
11        elif key == KEY_3: player_input_received = 3
12        elif key == KEY_4: player_input_received = 4
13        elif key == KEY_5: player_input_received = 5
14        elif key == KEY_6: player_input_received = 6
15        elif key == KEY_7: player_input_received = 7
16        elif key == KEY_8: player_input_received = 8
17        elif key == KEY_9: player_input_received = 9
18
19        if player_input_received != -1:
20            waiting_for_input = false
21            handle_player_card_choice(player_input_received)

```

4.4.3 Game loop — faza igrača

Na početku svog poteza igrač dobiva 3 energije i vuče jednu kartu. Petlja se ponavlja sve dok igrač ima karte i energiju, ili dok sam ne odluči završiti potez pritiskom na tipku 0.

```

1 func player_phase():
2     player.energy = 3
3     player.draw_card()
4
5     print("\n--- PLAYEROV POTEZ ---")
6     print("Player HP: ", player.hp, " | Enemy HP: ", enemy.hp)
7
8     prompt_player()
9
10
11 func prompt_player():
12     if player.hand.size() == 0:
13         print("Nemas vise karata u ruci.")
14         end_player_turn()
15         return
16
17     if player.energy == 0:
18         print("Nemas vise energije.")
19         end_player_turn()
20         return
21
22     print("\nEnergija: ", player.energy)
23     print("Tvoje karte:")
24     for i in range(player.hand.size()):
25         var c = player.hand[i]
26         print("  ", i + 1, ". ", c.naziv,
27             " | Attack: ", c.attack,
28             " | Cost: ", c.cost)
29     print("  0. Zavrshi potez")
30     print("Pritisni broj karte ili 0 za zavrsetek:")
31
32     player_input_received = -1
33     waiting_for_input     = true
34
35
36 func handle_player_card_choice(input: int):
37     if input == 0:
38         print("Zavrstavas potez.")
39         end_player_turn()
40         return
41
42     if input > player.hand.size():
43         print("Nema karte s tim brojem, pokusaj ponovo.")

```



```

44     prompt_player()
45     return
46
47     var card = player.hand[input - 1]
48
49     if card.cost > player.energy:
50         print("Nemas dovoljno energije! (treba ",
51             card.cost, ", imas ", player.energy, ")")
52         prompt_player()
53         return
54
55     player.hand.remove_at(input - 1)
56     player.energy -= card.cost
57     player_attack(card)
58     print("Odigrao si: ", card.naziv,
59         " | Attack: ", card.attack,
60         " | Ostalo energije: ", player.energy)
61
62     if check_game_over():
63         return
64
65     prompt_player()
66
67
68 func end_player_turn():
69     player_turn = false
70     enemy_phase()

```

Listing 8: Main.gd — player_phase() i prompt_player()

4.4.4 Game loop — faza protivnika

Protivnik automatski igra sve karte koje može platiti energijom, redom dok mu ne pones-tane energije ili karata.

```

1 func enemy_phase():
2     print("\n--- PROTIVNIKOV POTEZ ---")
3     enemy.energy = 3
4     enemy.draw_card()
5
6     var played_any = true
7     while played_any and enemy.hand.size() > 0 and enemy.energy >
8         0:
9         played_any = false
10
11     for i in range(enemy.hand.size()):
12         var card = enemy.hand[i]
13         if card.cost <= enemy.energy:
14             enemy.hand.remove_at(i)
15             enemy.energy -= card.cost
16             enemy_attack(card)
17             print("Enemy igra: ", card.naziv,

```

```

17         " | Attack: ", card.attack,
18         " | Ostalo energije: ", enemy.energy)
19     played_any = true
20     break
21
22     print("Player HP: ", player.hp, " | Enemy HP: ", enemy.hp)
23
24     if check_game_over():
25         return
26
27     player_turn = true
28     player_phase()

```

Listing 9: Main.gd — enemy_phase()

4.4.5 Napad i kraj igre

```

1 func player_attack(card: Card):
2     enemy.hp -= card.attack
3     if enemy.hp < 0:
4         enemy.hp = 0
5
6 func enemy_attack(card: Card):
7     player.hp -= card.attack
8     if player.hp < 0:
9         player.hp = 0
10
11 func check_game_over() -> bool:
12     if enemy.hp <= 0:
13         print("\n=== ", player_username, " POBJEDUJE! ===")
14         game_over = true
15         waiting_for_input = false
16         return true
17     elif player.hp <= 0:
18         print("\n=== ENEMY POBJEDUJE! ===")
19         game_over = true
20         waiting_for_input = false
21         return true
22     return false

```

Listing 10: Main.gd — napad i provjera kraja

4.4.6 Učitavanje podataka iz baze

```

1 func load_player_from_database():
2     var db = SQLite.new()
3     db.path = "user://game.db"
4     db.open_db()
5
6     var result = db.select_rows("korisnik", "", ["id", "username",
7         , "email"])

```

```

8     if result.size() > 0:
9         player_username = result[0][1]
10        player_email     = result[0][2]
11        print("Korisnik: ", player_username, " (" , player_email,
12              ")")
13    else:
14        print("Nema korisnika u bazi, koristim zadano ime.")
15
16    db.close_db()
17
18    func load_cards_from_database():
19        var db = SQLite.new()
20        db.path = "user://game.db"
21        db.open_db()
22
23        var deck_result = db.select_rows(
24            "deck",
25            "vlasnik_tip = 'korisnik' AND korisnik_id = 1",
26            ["id"]
27        )
28
29        if deck_result.size() == 0:
30            print("Nema decka za playera!")
31            db.close_db()
32            return
33
34        var deck_id = deck_result[0][0]
35        var dk      = db.select_rows(
36            "deck_karta",
37            "deck_id = " + str(deck_id),
38            ["karta_id", "kolicina"]
39        )
40
41        for dk_row in dk:
42            var karta_result = db.select_rows(
43                "karta",
44                "id = " + str(dk_row[0]),
45                ["id", "naziv", "attack", "cost"]
46            )
47            if karta_result.size() > 0:
48                var row = karta_result[0]
49                for j in range(dk_row[1]):
50                    var card = Card.new()
51                    card.id    = row[0]
52                    card.naziv = row[1]
53                    card.attack = row[2]
54                    card.cost  = row[3]
55                    player.deck.append(card)
56
57        db.close_db()

```

```

58     print("Učitano ", player.deck.size(), " karata iz player
        decka.")
59
60
61 func load_enemy_from_database():
62     var db = SQLite.new()
63     db.path = "user://game.db"
64     db.open_db()
65
66     var result = db.select_rows(
67         "protivnik", "",
68         ["id", "naziv", "hp", "difficulty"]
69     )
70
71     if result.size() > 0:
72         var row = result[0]
73         enemy.hp = row[2]
74         enemy.difficulty = row[3]
75         print("Protivnik: ", row[1],
76             " | HP: ", enemy.hp,
77             " | Tezina: ", enemy.difficulty)
78
79     var cards = db.select_rows("karta", "", ["id", "naziv", "
        attack", "cost"])
80     for row in cards:
81         var card = Card.new()
82         card.id = row[0]
83         card.naziv = row[1]
84         card.attack = row[2]
85         card.cost = row[3]
86         enemy.deck.append(card)
87
88     db.close_db()

```

Listing 11: Main.gd — load funkcije

Tijek igre

1. Igra se pokreće i učitava sve podatke iz SQLite baze podataka
2. Igrač i protivnik svaki dobivaju po 3 početne karte
3. **Playerov potez:** igrač dobiva 3 energije i vuče jednu kartu; pritiskom tipki 1–9 bira kartu za odigrati, a tipkom 0 završava potez
4. **Protivnikov potez:** protivnik automatski igra sve karte koje može platiti energijom
5. Koraci 3 i 4 ponavljaju se naizmjenično
6. Igra završava kada HP igrača ili protivnika padne na 0

Zaključak

Projekt prikazuje kako se može spojiti Godot Engine s relacijskom bazom podataka kako bi se dinamički upravljalo sadržajem igre. Implementirani su osnovni mehanizmi kartaške igre — upravljanje špilom, vučenje karata, sustav energije i naizmjenični potezi. Baza podataka omogućuje lako dodavanje novih karata, protivnika i korisnika bez mijenjanja koda igre.